International Journal of Computer Integrated Manufacturing

ISSN: 0951-192X (Print) 1362-3052 (Online) Journal homepage: http://www.tandfonline.com/loi/tcim20

# Automatic assembly sequence planning from STEP CAD files

C. Pan , S. S. Smith & G. C. Smith

**To cite this article:** C. Pan , S. S. Smith & G. C. Smith (2006) Automatic assembly sequence planning from STEP CAD files, International Journal of Computer Integrated Manufacturing, 19:8, 775-783, DOI: 10.1080/09511920500399425

**To link to this article:** http://dx.doi.org/10.1080/09511920500399425

Published online: 19 Feb 2007.

Submit your article to this journal

Article views: 83

View related articles

Citing articles: 6 View citing articles

Full Terms & Conditions of access and use can be found at
http://www.tandfonline.com/action/journalInformation?journalCode=tcim20

**Download by:** [University of Nebraska, Lincoln] **Date:** 17 September 2015, At: 03:23

*International Journal of Computer Integrated Manufacturing*, Vol. 19, No. 8, December 2006, 775–783

Taylor & Francis
Taylor & Francis Group

# Automatic assembly sequence planning from STEP CAD files

C. PAN†, S. S. SMITH*‡ and G. C. SMITH§

†Iowa State University, Department of Agricultural and Biosystems Engineering, Ames, IA 50011
‡Iowa State University, Virtual Reality Applications Center, Department of Agricultural and Biosystems Engineering, 2274 Howe Hall, Ames, IA 50011, USA
§CE Engineering, 5111 Schubert Street, Ames, IA 50014, USA

Downloaded by [University of Nebraska, Lincoln] at 03:23 17 September 2015

In recent years, many research studies related to theoretical and applied assembly sequence planning have been conducted. However, with most prior proposed assembly planning methods, users need to manually enter all or part of the geometrical information needed to find valid assembly sequence plans for a given product design. In this paper, we propose a fully automated assembly sequence planner, which directly extracts geometrical information from an input CAD file and then finds assembly sequences, which require a minimum or near-minimum number of assembly direction reorientations, for a given product design. To make the method compatible with different CAD software packages, the proposed assembly planner takes, as input, a CAD STEP (STandard for the Exchange of Product model data) file. The planner uses a powerful and efficient genetic algorithm to find assembly sequences with a minimum or near-minimum number of reorientations.

*Keywords*: Assembly planner; STEP; JSDAI; Genetic algorithm; Interference-free matrix

## 1. Introduction

During the lifecycle of a product, the product's assembly plan significantly affects both assembly process efficiency and assembly line design. As a result, many research studies related to theoretical and practical assembly sequence planning have been conducted. Prior assembly planning research can be classified into either graph-based or artificial intelligence-based methods.

Graph-based methods include feature mating (Kim and Wu 1990, Gu and Yan 1995, Anantha *et al.* 1996, Ling and Narayan 1996, Eng *et al.* 1999), AND/OR graphs (Homen de Mello and Sanderson 1990, 1991), connectivity graphs (Shpitalni *et al.* 1989), hierarchical partial-order graphs (Lee and Shin 1988), liaison diagrams (Bourjault 1984, De Fazio and Whitney 1987, Baldwin *et al.* 1991), precedence diagrams (Prenting and Battaglin 1964, Bullinger and Ammer 1984, Delchambre 1990, Yin *et al.* 2003), assembly constraint graphs (Wolter 1990) and interference graphs (De Floriani 1989).

Graph-based methods have been used extensively, because they guarantee that an optimal assembly sequence can be found, if one exists, for a given product structure and a given set of constraints. On the other hand, graph-based methods need a significant amount of computer memory space to store product assembly and subassembly sequence information. Graph-based assembly planners are also somewhat limited, with respect to product size, since runtimes for graph-based algorithms are usually exponential with respect to the number of components in the assembly (Yin *et al.* 2003).

As a result, many recent research studies have explored using artificial intelligence methods for assembly sequence planning. Artificial intelligence-based methods generally use either genetic algorithms (Sanderson *et al.* 1990, Bonneville *et al.* 1995, Sebaaly and Fujimoto 1996, Chen 1998, De Lit *et al.* 2001, Smith *et al.* 2001, Smith and Smith 2002, 2003) or knowledge-based expert systems (Zha *et al.* 1998, 1999, Zha 2000, Zha and Du 2002).

Both graph-based methods and artificial intelligence-based methods depend upon assembly models, which

*Corresponding author. Email: sssmith@iastate.edu

*International Journal of Computer Integrated Manufacturing*
ISSN 0951-192X print/ISSN 1362-3052 online © 2006 Taylor & Francis
http://www.tandf.co.uk/journals
DOI: 10.1080/09511920500399425

Downloaded by [University of Nebraska, Lincoln] at 03:23 17 September 2015

contain geometrical information such as assembly structure and assembly constraints. Assembly models are usually stored in computer-aided design (CAD) files or databases. With most prior assembly planners, user interaction is needed during the assembly sequence planning process to manually enter assembly feature or spatial constraint information or to interactively answer questions asked by the assembly planner.

To automate the assembly planning process more fully and, thus, to reduce the time required to find assembly plans for any given product, automated methods are needed for directly extracting and analysing geometrical information and spatial constraints from the CAD files of designed products. In other words, automated assembly planners must be able to communicate with product designs directly via CAD files without additional manual interaction. However, prior studies have not adequately addressed the problem of fully automating the assembly sequence planning process.

Hoffman (1990) presented an assembly planner which takes, as input, boundary representation (B-Rep) models of components and then generates sequences of operations for assembling a product. Hoffman's planner searches feasible sequences based on information concerning directions in which components are free to move and how far they can move before colliding with other components. During the search for feasible sequences, a line-up agent is involved, which lines up component feature points to hypothesize potentially useful component assembly motions. Given component feature points and B-Rep CAD model data, the planner automatically finds feasible assembly sequences for a product, without any manual guidance during the search. However, in Hoffman's method, both feature points and B-Rep CAD model data must be manually supplied, as input, to the assembly planner. In addition, the planner does not take any criteria into consideration for selecting preferred assembly sequences from the set of all feasible sequences.

Laperriere and ElMaraghy (1991) proposed a method for automatically generating assembly sequences for robotic assembly processes. Their method includes an interface by which a user must manually enter geometric product feature information into a product feature-base (database). In addition, their method requires the user to manually create relation diagrams to represent physical connections and spatial constraints between components of the product. With the given input information, the planner automatically finds a single assembly sequence for a robotic assembly process. Laperriere and ElMaraghy's method requires a lot of user interaction, both to create the input feature base and, during execution, to create relation diagrams. Thus, their method is not a fully automatic assembly sequence planner, although their computer program can automatically find a valid assembly sequence from manually entered input information.

Venuvinod (1993) proposed two algorithms for rapidly constructing a disassembly tree, for automatically generating assembly sequences for products composed of three-dimensional (3D) polyhedral components modelled in boundary representation format. One of their algorithms determines mating information between pairs of mating components, based on product B-Rep information. Their other algorithm determines the possibility of disassembling a given component from a subassembly, also based upon product B-Rep information. Their study provides a foundation for automatically finding assembly sequences for 3D polyhedral products. However, their algorithms do not fully implement or automate the assembly sequence planning process.

Mascle (1998) proposed an approach using disassembly analysis to study the problems related to automatic assembly sequence planning. Mascle's approach automatically extracts liaisons between components from a B-Rep CAD model of a mechanical product and describes the liaisons in matrices which contain descriptions of half degrees of the liaisons. The approach automatically determines a stable subassembly and the degrees of freedom for the subassembly during the disassembly process. However, the author did not describe his method for extracting liaisons from B-Rep CAD models. In addition, the study did not describe a complete process for generating disassembly/assembly sequences.

Mok *et al.* (2001) proposed a method for automatically generating assembly sequences and part-mating operations from CAD data contained in STEP files. However, they did not introduce detailed methods for extracting the geometrical information from STEP files needed to construct assembly models, and they did not consider methods for selecting optimal assembly sequences with respect to a given set of constraints. Mok *et al.* proposed a simple assembly sequence generation method based upon ordering assembly parts by part extrema in a single-axis direction only.

Zha and Du (2002) proposed a STEP-based method for concurrent integrated design and assembly planning. Zha and Du described, in detail, STEP schema and protocols for defining assemblies. Their research focused on using STEP to describe the prototype of an integrated design and assembly system. They did not propose methods for extracting geometrical product information from STEP CAD files or for selecting preferred assembly sequences with respect to certain criteria.

Prior assembly planning studies have proposed using matrices to represent interference relationships among components for assembly planning. Dini and Santochi (1992) used $n \times n$ interference matrices to represent interference relationships between the $n$ parts in an assembly model, with one matrix $A_k$ for each assembly direction $k$. For Dini and Santochi's interference matrix $A_k$, $a_{ij} = 1$, if part $i$ interferes with part $j$ when moving along the

Downloaded by [University of Nebraska, Lincoln] at 03:23 17 September 2015

$k$ direction into assembled position with part $j$ already in assembled position. Smith *et al.* (2001) defined six $n \times n$ interference-free matrices to represent interference relationships between the $n$ parts in an assembly model, for the six principal axis directions. However, both Dini and Santochi and Smith's matrices were manually generated by a user, based upon manual analysis of the CAD assembly model of a product. However, manual generation of constraint matrices takes a lot of time and may introduce errors. To improve the efficiency and accuracy of assembly planning, automatic generation of interference matrices is important.

The objective of this paper is to present a fully automated assembly sequence planner. The proposed fully automated assembly planner can automatically (without any human intervention) extract geometrical information directly from STEP CAD files, automatically generate interference-free matrices to represent interference relationships between assembly components and then automatically generate assembly sequences which require a minimum or near-minimum number of assembly direction reorientations. Using CAD STEP files provides an advantage over other input data formats, since many design, manufacturing, and assembly planning software applications support STEP file input and output capabilities. Thus, the proposed assembly planner is compatible with many different CAD application packages.

## 2. Extracting geometrical information from STEP CAD files using JSDAI

A fully automated assembly planner must be able to extract product geometrical information directly from CAD applications or CAD files and then automatically analyse the extracted information to create product assembly models. An assembly planner that can extract geometrical information from STEP CAD files can be used with many different CAD applications, since most popular CAD applications have STEP input and output capabilities. Therefore, the proposed assembly planner was designed to extract product assembly model information from STEP files.

### 2.1. *Introduction to STEP*

STEP (STandard for the Exchange of Product model data) is a family of ISO (International Standards Organization) standards established for representing and exchanging industrial product data between CAX/PDM (CAX, an inclusive summary term for various computer aided technologies e.g. CAO, CAM, CAE, etc.; PDM, product data mangement) applications.

Part 21 of STEP (physical file format) defines the format and structure of a STEP-format file. In addition, the format of mechanical part and assembly data contained in a STEP file must conform to Part 203 of STEP (ISO 10303-203), which is AP203 (Application Protocol 203) (Smith *et al.* 2002).

Part 22 of STEP (SDAI, the Standard Data Access Interface of STEP) is an abstract specification that defines the functionality of an API (application programming interface) or a toolbox for simplifying STEP-capable application program development (SDAI Overview 2003).

By implementing a complete Java programming language binding to SDAI, application developers can develop a Java program which can directly access data in a STEP Part 21 file. For the study conducted, an application program interface–JSDAI, developed by LKSoft (http://www.lksoft.com), was used to add STEP file access capabilities to the proposed fully automated assembly planner. The JSDAI packages were imported into a Java application, which was developed for directly reading geometrical data from STEP Part 21 files, without translating the STEP files into any other data file format.

### 2.2. *Extracting local part coordinates from STEP files*

Four classes in the JSDAI package are needed for extracting local part coordinates from a STEP Part 21 file: SdaiSession, SdaiRepository, SdaiModel, and EntityExtent.

The SdaiSession class is used for initializing and terminating any SDAI activity, completing data transfer transactions between the program and the STEP Part 21 file, and dynamically creating new repositories ('JSDAI V 2.3: Core API' 2003). An SdaiSession class object creates a new repository for the STEP Part 21 file, in a user-defined directory and opens a tunnel between the repository and the STEP Part 21 file, for completing data transfer transactions.

The SdaiRepository class is a physical container for persistent storage of SchemaInstances, SdaiModels, and application instances ('JSDAI V 2.3: Core API' 2003). Persistent storage of SdaiModels and SchemaInstances in an SdaiRepository is realized through special directories containing binary data files. By default, the directories are created in a special repositories directory within the local file system. Users specify the repositories directory by setting the 'repositories' property in the jsdai.properties configuration file. An SdaiRepository class object can obtain all of the SdaiModels from an imported STEP Part 21 file or create a STEP Part 21 file that stores all the data contained in the repository.

The SdaiModel class is used for grouping entity instances that belong to the same EXPRESS schema. Each SdaiModel is based upon one EXPRESS schema, called the underlying schema. All entity instances that can appear in an SdaiModel should be defined or declared in the EXPRESS schema underlying that particular SdaiModel. Every entity instance within an SdaiSession must belong to an SdaiModel.

The EntityExtent class groups all entity instances within an SdaiModel into folders. There is one EntityExtent instance for each entity data type defined or declared in an

EXPRESS schema. A folder corresponding to an entity data type contains all instances of that particular entity data type within the model and also contains all instances of that particular entity data type's subtypes within the same model. As a result, each folder represents an EntityExtent instance.

A STEP Part 21 file includes two sections: a header section and a data section. The data section of a STEP Part 21 file represents one SdaiModel. All entity instances in the data section are grouped into folders corresponding to different entity types within the SdaiModel.

In the developed JAVA program, to start extracting local part coordinates, an object for each of the classes – SdaiSession, SdaiRepository, SdaiModel, and Entity Extent – was initialized. Then, the getPopulatedFolders() method of SdaiModel class was used to return a set of EntityExtent instances corresponding to each entity data type defined or declared in the EXPRESS schema for the given SdaiModel.

All the topological information and all the coordinates of a part were obtained from the EntityExtent instance named 'topological_representation_item'. All the elements in the B-rep (Boundary representation) of each part were contained within the folder as the entity type and subtype instances. By reading and following indices from the instances, and from each instance's components, the tree-like B-rep data structure of a part was extracted from the STEP Part 21 file.

In the developed program, three arrays were used to store the geometrical entities contained in the B-rep data structure of a part. The geometrical relationships between entities in the B-rep data structure can be represented as shown in figure 1. In figure 1, entities on the face-axis represent all faces of a part. A one-dimensional (1D) array was used to save all part faces. Entities on the face–edge plane represent all part edges. A two-dimensional (2D) array was used to save all part edges. Entities in the face–edge–vertex space represent all part vertices. A 3D array was used to save all part vertices. For example, if edge 23 represents the third edge on the second face of a part, edge 23 would be saved in a 2D array. As shown in figure 1, if vertex 02*n* represents the *n*th vertex of the second edge on the 0th face of the part, vertex 02*n* would be stored as one of the elements in the 3D array.

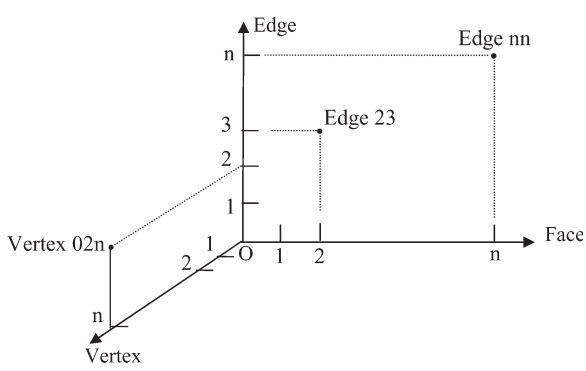


Figure 1. Relationships between all geometrical entities.

In a STEP file, within the B-rep structure of a part, every edge is repeated twice (belongs to two faces) and every vertex is repeated six times (belongs to three faces and, on each face, belongs to two edges). Therefore, in the 2D array, each element was recorded twice; in the 3D, each element was recorded six times.

### 2.3. *Transforming parts into a uniform coordinate space*

By analysing assembly models stored in STEP files, the investigators determined that the coordinates of each part in an assembly model are given with respect to a local coordinate space for that particular part. In STEP files, a default assembly coordinate space is defined, which only matches the coordinate space for the first part in the assembly model. As a result, to analyse interference relationships between parts during assembly, part coordinates, for all parts except the first part in the STEP file, need to be transformed into the assembly coordinate space. In the proposed method, transformation information, contained within the STEP file, for each part in an assembly model is extracted and used to transform the part coordinates into the assembly coordinate space.

By creating and analysing several different STEP assembly models, the investigators also determined that the transformation information stored within the STEP file, for each part, includes two Cartesian points for origins (new and original origin coordinates), two vectors for axis directions (new and original axis directions), and two vectors for reference directions (new and original reference directions). The transformation information for all parts in the assembly model can be extracted from the EntityExtent instance named 'item_defined_transformation'. Each item of the entity type 'item_defined_transformation' represents the transformation information of a single assembly model part. Each item has two attributes: 'transform_item_1' and 'transform_item_2'. The 'transform_item_1' attribute contains the part's new origin coordinates, new axis direction, and new reference direction. The 'transform_item_2' attribute contains the part's original origin coordinates, original axis direction, and original reference direction.

New and original origin coordinates determine the translation needed to transform a part from its local coordinate space into the assembly coordinate space. New and original axis and reference directions determine the rotation needed to transform a part from its local coordinate space into the assembly coordinate space. Therefore, in the proposed method, translation and rotation matrices are found for transforming each part in the assembly model into the assembly coordinate space. Each part vertex is then transformed into the assembly coordinate space, using matrix multiplication methods.

Downloaded by [University of Nebraska, Lincoln] at 03:23 17 September 2015

Downloaded by [University of Nebraska, Lincoln] at 03:23 17 September 2015

## 3. Determining interference relationships between parts

Since most mechanical components (in automated assembly processes) are assembled along the principal axes (Chen 1998), the proposed assembly planner considers interference relationships between parts, during assembly, in six principal axis directions ($+x$, $-x$, $+y$, $-y$, $+z$, and $-z$). The proposed planner uses geometrical 3D-to-2D projection methods, along the six principle axis directions, to determine interference relationships between parts during assembly. If the projections of two parts along a given axis direction overlap, the two parts may collide when one of the parts moves in the given direction towards or past the other part.

The proposed assembly planner simplifies the task of determining 3D interference relationships into determining 2D interference relationships, by using part projection methods along the six principle axis directions. The planner determines 2D projections of 3D parts, in a given direction and onto a given 2D projection plane, from 3D part vertex coordinates, the projection direction, and the geometrical definition of the projection plane. From 2D part projections, 3D interference relationships between parts, during assembly operations, are then determined.

In the proposed method, the detected interference relationships among components for an assembly model are stored in six interference-free matrices, as defined by Smith *et al.* (2001). For example, assume that the interference-free matrix of a three-part assembly structure for the $+y$ direction is

$$\begin{array}{c} \text{Part} \\ 1 \\ 2 \\ 3 \end{array} \begin{array}{c} \begin{array}{ccc} 1 & 2 & 3 \end{array} \\ \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \end{array} \quad \text{(Interference-free matrix for } +y \text{ direction)}$$

This matrix indicates that, part 2 will collide with part 1 along the $+y$ direction, if part 1 has already been assembled. On the other hand, part 1 will not collide with part 2 along the $+y$ direction, if part 2 has already been assembled.

The interference-free matrices, which are automatically generated, are then used in the downstream assembly sequence planning process – selecting geometrically feasible assembly sequences which require a minimum or near-minimum number of assembly direction reorientations.

## 4. Finding assembly sequences that require a minimum or near-minimum number of assembly direction reorientations for a given assembly model

The proposed fully automated assembly planner integrates and improves upon the genetic algorithm-based assembly planner proposed by Smith and Smith (2002). Specifically, the proposed assembly planner uses the genetic algorithm-based assembly planner developed by Smith and Smith to find geometrically feasible assembly sequences, which require a minimum or near-minimum number of assembly direction reorientations, from a given set of interference-free matrices. However, the proposed fully automated assembly planner automates the process of generating interference-free matrices directly from CAD STEP files. The new assembly planner uses Smith and Smith's genetic algorithm-based assembly planner, since their planner can find minimum or near-minimum reorientation assembly sequences more reliably and more quickly than prior genetic algorithm-based assembly planners.

Using the new integrated assembly planner, valid assembly sequences are found which meet geometrical assembly constraints contained within the interference-free matrices. Then, evaluation criteria, such as number of reorientations, number of fixtures, number of tool changes, or part stability during assembly, can be used to select preferred sequences from the set of all valid sequences found.

In this paper, the planner used number of assembly direction reorientations as the evaluation criterion for selecting preferred assembly sequences. The fitness function used to evaluate geometrically feasible assembly sequences was: $\text{fitness}(s) = 2N_{\max} - n$, where $N_{\max}$ is the total number of parts in the assembly, and $n$ is the number of reorientations (or tool changes) required during assembly. For a geometrically unfeasible sequence, $\textit{fitness}(s) = N_{\max}/2$.

For the new, fully automated, assembly planner, input is a STEP CAD file and output is a set of geometrically feasible assembly sequences, which require a minimum or near-minimum number of assembly direction reorientations, along with the corresponding assembly direction for each part.

## 5. Implementation

### 5.1. *Case study 1*

Figure 2 shows an example model used for testing the proposed assembly planner. The model includes four parts. Part 1 has three through holes into which, during assembly, parts 2, 3, and 4 are inserted. Part 2 must be inserted in a different assembly direction than parts 3 and 4. Therefore, at least one assembly direction reorientation is required to completely assemble the model. The model was created using Pro/Engineer and saved in STEP format.

Figure 3 shows a process flowchart for the fully automated assembly planner. For the given model, after the assembly planner opened the STEP file for the model, the planner extracted the original coordinates of each part in the part's local coordinate system. The planner then extracted information needed to transform each part from its local coordinate system to the assembly coordinate system. After extracting both the original coordinates of each part and the transformation information for each part, the assembly planner calculated the coordinates of each part in the assembly coordinate system. With the unified coordinates for each

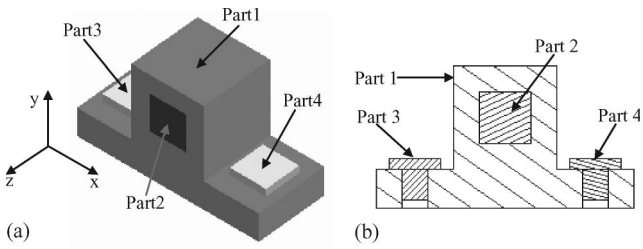


Figure 2. (a) 3D assembly model. (b) Section view of the assembly.

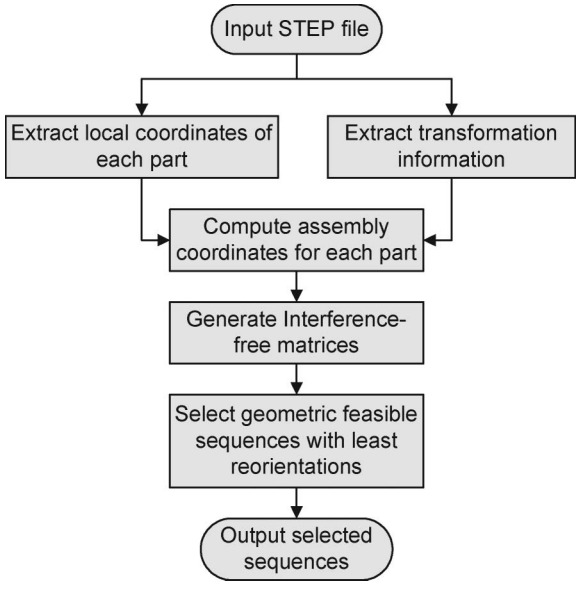


Figure 3. Process flowchart for the assembly planner.

part, the planner generated interference-free matrices and searched for geometrically feasible sequences, which require a minimum or near-minimum number of assembly direction reorientations, based on the generated matrices.

For the given test case, the assembly planner automatically transformed all parts into a unique assembly coordinate system and then created the following six interference-free matrices

$$+x \quad \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix} \qquad -x \quad \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix} \qquad +y \quad \begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

$$-y \quad \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} \qquad +z \quad \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \qquad -z \quad \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

After deriving the six interference-free matrices, the assembly planner automatically searched for geometrically feasible sequences which required a minimum or near minimum number of reorientations. Four resulting output assembly sequences, which were automatically generated from the input STEP file for the assembly model, are shown below.

| | |
|---|---|
| Sequence (2, 1, 4, 3) | assembly directions $(\pm z, \pm z, -y, -y)$ |
| Sequence (2, 1, 3, 4) | assembly directions $(\pm z, \pm z, -y, -y)$ |
| Sequence (1, 2, 3, 4) | assembly directions $(\pm z, \pm z, -y, -y)$ |
| Sequence (1, 3, 4, 2) | assembly directions $(-y, -y, -y, \pm z)$ |

During the search for assembly sequences, the assembly planner allows users to set genetic algorithm population size. Smith *et al.* (2001) recommends setting assembly planner population size to approximately the number of parts in the assembly. For the four-part model in figure 2, a population size was set to four assembly sequences, and the assembly planner was run 20 times.

Two termination criteria for the search were used: a limit of 500 generations or when average population fitness reached a set level. Searching stopped when either of the two termination criteria was met. Average population fitness was first set to zero reorientations, but the search terminated after 500 generations without finding four zero reorientation assembly sequences. Average population fitness was then set to one reorientation, and the search terminated after successfully finding four one-reorientation sequences in all 20 runs. During the search process, assembly sequences in the population were not required to be unique.

For 20 runs, the average final fitness was 7.00 and the average run time was 203.98 s (Including overall runtime for the entire process: extracting information, calculating interference-free matrices, and finding assembly sequences). The planner found different one-reorientation assembly sequences in different runs. The output assembly sequences given above are from the last of the 20 runs. From the given output, several one-reorientation assembly sequences exist. One reorientation is the minimum number of assembly direction reorientations for the given assembly model.

### 5.2. *Case study 2*

Figure 4 shows CAD model and a photograph for a subassembly from a power controller, which was used for case study 2. For the given model, the proposed assembly planner automatically generated the following interference-free matrices, using the projection method:

Downloaded by [University of Nebraska, Lincoln] at 03:23 17 September 2015

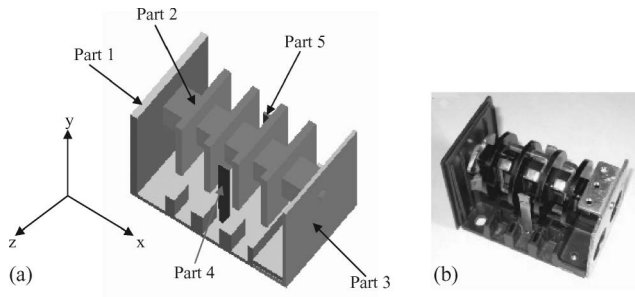


Figure 4. (a) Simplified controller model. (b) Real model of the controller.

$$+x \quad \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix} \quad -x \quad \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \quad +y \quad \begin{bmatrix} 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

$$-y \quad \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 0 \end{bmatrix} \quad +z \quad \begin{bmatrix} 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 \end{bmatrix} \quad -z \quad \begin{bmatrix} 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 \end{bmatrix}$$

For the controller model, a population size of five was used, and the assembly planner was run 20 times. Two termination criteria for the search were used: a limit of 500 generations and a required maximum fitness value for at least one assembly sequence in the final population. The search was terminated when either of the termination criteria was met. The maximum fitness value for at least one assembly sequence was first set to 10 (zero assembly direction reorientations), but the search failed to find a zero reorientation assembly sequence after 500 genetic algorithm generations. The maximum value for at least one assembly sequence was then set to 9 (one assembly direction reorientation) and the search terminated successfully in all 20 runs. During the search process, assembly sequences in the population were not required to be unique.

The assembly planner was run 20 times to determine the average number of generations and the average runtime required to find geometrically feasible sequences which required a minimum or near minimum number of assembly direction reorientations, for the given example. The genetic algorithm-based assembly planner uses a stochastic search technique, and, therefore, may not find a global optimal solution for any given assembly structure. However, in practice, near-optimal solutions may be acceptable. On the other hand, genetic algorithm-based assembly planners, in general, run much more quickly than graph-searching assembly planners.

Actual assembly planner results for the first of the 20 runs are given below. The initial and final populations for the given run are shown, along with fitness values for each member (assembly sequence) in the initial and final populations. The initial population was generated automatically, using a random number generator.

For the given example, at least one reorientation (fitness = 9) is needed to completely assemble the device. A fitness value of $(N_{\max}/2) = (5/2) = 2.5$ indicates that a given assembly sequence is invalid; the device cannot be assembled using the given assembly sequence, since the given assembly order leads to a violation of geometric constraints. All initial generation assembly sequences are invalid. Therefore, each initial assembly sequence has a fitness value of 2.5.

Initial generation:

(5, 4, 1, 2, 3) Fitness = 2.5
(5, 3, 2, 4, 1) Fitness = 2.5
(5, 2, 1, 3, 4) Fitness = 2.5
(3, 5, 4, 2, 1) Fitness = 2.5
(4, 3, 1, 5, 2) Fitness = 2.5

Final generation:

(1, 2, 3, 4, 5) Fitness = 9.000000
(2, 1, 3, 5, 4) Fitness = 8.000000
(2, 1, 3, 4, 5) Fitness = 8.000000
(2, 1, 3, 5, 4) Fitness = 8.000000
(2, 1, 3, 4, 5) Fitness = 8.000000

Average fitness for the first generation is 2.5. Average fitness for the last generation is 8.2. The runtime was 0.05 s (This time does not include extracting information and generating matrices). For the last generation, the assembly sequence with the highest fitness value (9) requires one assembly direction reorientation, which is a known minimum with respect to number of assembly direction reorientations, for the given test case. Thus, for case study 2, the proposed fully automated assembly planner, without human intervention, found an assembly sequence (1, 2, 3, 4, 5) for the given test case. The assembly planner also reported the allowed assembly directions for each part in the given assembly sequence:

Part 1: $\{+x, -x, +y, -y, +z, -z\}$
Part 2: $\{-x\}$
Part 3: $\{-x\}$
Part 4: $\{-y\}$
Part 5: $\{-y\}$

Downloaded by [University of Nebraska, Lincoln] at 03:23 17 September 2015

The allowed assembly directions show that to assemble the subassembly from the power controller in the given order, with only one reorientation, requires that parts 1, 2, and 3 be assembled in the $-x$ direction, and that parts 4 and 5 be assembled in the $-y$ direction.

Assembly planner results for the last of the 20 runs are given below.

Initial generation:

(2, 5, 4, 3, 1) Fitness = 2.5
(4, 1, 5, 3, 2) Fitness = 2.5
(2, 4, 5, 1, 3) Fitness = 2.5
(1, 2, 3, 4, 5) Fitness = 9.0
(3, 2, 1, 5, 4) Fitness = 9.0

Final generation:

(3, 2, 1, 5, 4) Fitness = 9.000000
(3, 2, 1, 5, 4) Fitness = 9.000000
(1, 2, 3, 4, 5) Fitness = 9.000000
(1, 2, 4, 5, 3) Fitness = 8.000000
(1, 2, 5, 3, 4) Fitness = 7.000000

Average fitness for the first generation is 5.1, and average fitness for the last generation is 8.4. The runtime for sequence searching was 0.01 s (This time does not include extracting information and generating matrices). By chance, two of the randomly generated assembly sequences in the first generation require only one reorientation. However, the assembly planner improved overall population fitness from 5.1 to 8.4, within 0.01 s. The planner also found geometrically feasible sequences which require two reorientations (Fitness = 8.0) and three reorientations (Fitness = 7.0).

For the one-reorientation assembly sequence (3, 2, 1, 5, 4), the assembly planner also reported the allowed assembly directions:

Part 3: $\{+x, -x, +y, -y, +z, -z\}$
Part 2: $\{+x\}$
Part 1: $\{+x\}$
Part 5: $\{-y\}$
Part 4: $\{-y\}$

The results show that one reorientation is needed to complete the given assembly sequence. The first three parts should be assembled in the $+x$ direction, and the last two parts should be assembled in the $-y$ direction.

For all 20 runs, the average final fitness was 8.33 and the average runtime was 10.21 s (This is overall implementation time of the planner for each run). The planner, in all 20 runs, found more than one geometrically feasible one-reorientation sequence.

## 6. Conclusion

This paper presents a fully automated assembly sequence planner. The proposed fully automated assembly planner improves the capabilities of Smith and Smith's genetic algorithm-based assembly planner (2002) by directly extracting geometric information from STEP CAD files, analysing the geometric information to determine geometric assembly constraints, and automatically generating interference-free matrices to represent spatial constraints between components during assembly operations.

The fully automated assembly planner eliminates the need for human interaction or intervention to generate necessary geometric input information for assembly planning, which was needed with previous assembly planners. The developed assembly planner takes a STEP CAD file as input and automatically outputs geometrically feasible assembly sequences which require a minimum or near-minimum number of assembly direction reorientations, for the given assembly model, along with assembly directions for each part.

As a result, the proposed fully automated assembly planner improves the efficiency of assembly sequence planning by completely eliminating the need for manual input during the assembly sequence planning process. Since STEP is an international standard, most CAD software applications provide STEP file input and output capabilities. As a result, the proposed assembly planner can be used to analyse assembly models from most CAD applications. Users simply need run the assembly planner and import the STEP CAD file that they would like to analyse.

The assembly planner automatically finds geometrically feasible assembly sequences which require a minimum or near-minimum number of assembly direction reorientations for the input assembly model, along with allowed assembly directions for each part in the output assembly sequences. The assembly planner also allows users to set search and search termination criteria, such as population size, maximum allowed runtime, maximum fitness required within the final generation, required average final fitness for the final generation, or whether the final generation should contain only unique assembly sequences.

In this paper, only the number of assembly direction reorientations was taken as a search criterion. However, during assembly, subassembly stability at each assembly step is also an important factor. The current developed assembly planner can also consider stability as a search criterion, using stability matrices as input, in addition to interference-free matrices. However, the developed planner cannot yet automatically generate the required stability matrices directly from CAD files. Thus, in future studies,

Downloaded by [University of Nebraska, Lincoln] at 03:23 17 September 2015

methods need to be developed for automatically generating stability matrices from STEP assembly models.

## References


Anantha, R., Kramer, G.A. and Crawford, R.H., Assembly modeling by geometrical constraint satisfaction. *Comput. Aided Des.*, 1996, **28**, 707–722.

Baldwin, D.F., Abell, T.E., Lui, M.M., De Fazio, T.L. and Whitney, D.E., An integrated computer aid for generating and evaluating assembly sequences for mechanical products. *IEEE Trans. Robot. Automn*, 1991, **7**, 78–94.

Bonneville, F., Perrard, C. and Henrioud, J.M., A genetic algorithm to generate and evaluate assembly plans. *Proceedings of the IEEE Symposium on Emerging Technology and Factory Automation*, 1995, pp. 231–239, Paris, France, Piscataway, NJ, USA.

Bourjault, A., Contribution a une Approche Methodologique de L'Assemblage Automatise: Elaboration Automatique des Sequences Operatoires. These d'Etat, Universite de Franche-Comte, Besancon, France, 1984.

Bullinger, H.J. and Ammer, E.D., Computer aided depicting of precedence diagrams – a step towards efficient planning in assembly. *Comput. Ind. Engng*, 1984, **8**, 165–169.

Chen, S.F., Assembly planning – a genetic approach. 1998 ASME *Design Engineering Technical Conferences*, 13–16 September 1998, Paper number DETC98/DAC-5798.

De Fazio, T.L. and Whitney, D.E., Simplified generation of all mechanical assembly sequences. *IEEE J. Robot. Automn*, 1987, **3**, 640–658.

De Floriani, G.N., A graph model for face-to-face assembly. *Proceedings of the IEEE International Conference on Robotics and Automation*, 1989, Vol. 1, pp. 75–78, Scottsdale, AZ, USA, Piscataway, NJ, USA.

Delchambre, A., A pragmatic approach to computer-aided assembly planning. *Proceedings of the IEEE International Conference on Robotics and Automation*, 1990, pp. 1600–1605, Cincinnati, OH, USA, Los Alamitos, CA, USA.

De Lit, P., Latinne, P., Rekiek, B. and Delchambre, A., Assembly planning with an ordering genetic algorithm. *Int. J. Prod. Res.*, 2001, **39**, 3623–3640.

Dini, G. and Santochi, M., Automated sequencing and subassembly detection in assembly planning. *Annals CIRP*, 1992, **41**, 1–4.

Eng, T.-H., Ling, Z.K., Olson, W. and McLean, C., Feature-based assembly modeling and sequence generation. *Comput. Ind. Engng*, 1999, **36**, 17–33.

Gu, P. and Yan, X., CAD-directed automatic assembly sequence planning. *Int. J. Prod. Res.*, 1995, **33**, 3069–3100.

Hoffman, R., Assembly planning for B-Rep objects. *Proceedings of the Rensselaer 2nd International Conference Computer Integrated Manufacturing*, New York, May, 1990, pp. 314–321.

Homen de Mello, L.S. and Sanderson, A.C., A correct and complete algorithm for the generation of mechanical assembly sequences. *IEEE Trans. Robot. Automn*, 1991, **7**, 228–240.

Homen de Mello, L.S. and Sanderson, A.C., AND/OR graph representation of assembly plans. *IEEE Trans. Robot. Automn*, 1990, **6**, 188–199.

Homen de Mello, L.S. and Sanderson, A.C., Representation of mechanical assembly sequences. *IEEE Trans. Robot. Automn*, 1991, **7**, 211–227.

JSDAI V 2.3: Core API. Retrieved August 10, 2003, from http://www.lksoft.com/jsdai_coreapi.php

Kim, M.G. and Wu, C., A formal part mating model for generating compliance control strategies of assembly operations. *IEEE Conference on Systems, Man and Cybernetics*, 4–7 November 1990, Los Angeles, CA, USA, Piscataway, NJ, USA.

Laperriere, L. and ElMaraghy, H.A., Automatic generation of robotic assembly sequences. *Int. J. Adv. Manuf. Technol.*, 1991, **6**, 299–316.

Lee, S. and Shin, Y.G., Automatic construction of assembly partial order graphs. *International Conference on Computer Integrated Manufacturing*, 23–25 May 1988.

Ling, Z.K. and Narayan, S., Feature representation and recognition using the object oriented concept and heuristics. *Integ. Comput. Aided Engng*, 1996, **3**, 212–224, Troy, NY, USA, New York, NY, USA.

Mascle, C., Automatic a priori, a posteriori or appropriate determination of subassemblies. *Int. J. Prod. Res.*, 1998, **36**, 1001–1021.

Mok, S., Ong, K. and Wu, C.H., Automatic generation of assembly instructions using STEP. *IEEE International Conference on Robotics and Automation*, 21–26 May 2001, Seoul, Korea.

Prenting, T.O. and Battaglin, R.M., The precedence diagram: A tool for analysis in assembly line balancing. *J. Ind. Engng*, 1964, **XV**, 208–213.

Sanderson, A.C., de Mello, L.S.H. and Zhang, H., Assembly Sequence planning. *AI Mag.*, 1990, **11**, 62–81.

SDAI Overview. Available online at: http://www.lksoft.com/sdai_overview. php (accessed 10 August 2003).

Sebaaly, M.F. and Fujimoto, H., A genetic planner for assembly automation. *Proceedings of the ICEC*, 1996, pp. 401 – 406, Nagoya, Japan, Piscataway, NJ, USA.

Shpitalni, M., Elber, G. and Lenze, E., Automatic assembly of three dimensional structures via connectivity graphs. *Annals CIRP*, 1989, **38**, 25–28.

Smith, G.C. and Smith, S.S.-F., An enhanced genetic algorithm for automated assembly planning. *Robot. Comput. Integr, Manuf.*, 2002, **18**, 355–364.

Smith, G. and Smith, S.S.-F., Automated initial population generation for genetic assembly planning. *Int. J. Comput. Integ. Manuf.*, 2003, **16**, 219–228.

Smith, G.C., Smith, R.A. and Smith, S.S.-F., STEP for Product Assembly (Selected paper), *2002 NAIT Convention,* 2002, pp. 193–202.

Smith, S.S.-F., Smith, G.C. and Liao, X., Automatic stable assembly sequence generation and evaluation. *J. Manuf. Systems*, 2001, **20**, 225–235.

Venuvinod, P.K., Automated analysis of 3-D polyhedral assemblies: assembly directions and sequences. *J. Manuf. Systems*, 1993, **12**, 246–252.

Wolter, J.D., On the automatic generation of assembly plans. *Proceedings of the IEEE International Conference on Robotics and Automation*, 1990, Vol. 1, pp. 62–68, Scottsdale, AZ, USA, Piscataway, NJ, USA.

Yin, Z.P., Ding, H., Li, H.X. and Xiong, Y.L., A connector-based hierarchical approach to assembly sequence planning for mechanical assemblies. *Comput. Aided Des.*, 2003, **35**, 37–56.

Zha, X.F., An object-oriented knowledge based Petri net approach to intelligent integration of design and assembly planning. *Artif. Intell. Engng*, 2000, **14**, 83–112.

Zha, X.F. and Du, H., A PDES/STEP-based model and system for concurrent integrated design and assembly planning. *Comput. Aided Des.*, 2002, **34**, 1087–1110.

Zha, X.F., Lim, S. and Fok, S.C., Integrated knowledge-based assembly sequence planning. *Int. J. Advd. Manuf. Technol.*, 1998, **14**, 50–64.

Zha, X.F., Lim, S. and Fok, S.C., Development of expert system for product design and planning for assembly. *Int. J. Advd. Manuf. Technol.*, 1999, **15**, 153–162.


Downloaded by [University of Nebraska, Lincoln] at 03:23 17 September 2015